COMPUTER PROGRAMMING ASSIGNMENT

1: Write a program to:

- Take 10 integer inputs in an array.
- Find and display: Maximum value Minimum value Average

```
#include <iostream>
using namespace std;

int main() {
    int arr[10];
    int sum = 0;

    // Taking input
    cout << "Enter 10 integers:\n";
    for(int i = 0; i < 10; i++) {
        cin >> arr[i];
        sum += arr[i]; // adding for average
    }

    int max = arr[0], min = arr[0];

    // Finding max and min
```

```
    for(int i = 1; i < 10; i++) {
        if(arr[i] > max)
            max = arr[i];
        if(arr[i] < min)
            min = arr[i];
    }

    // Display results
    cout << "Maximum: " << max << endl;
    cout << "Minimum: " << min << endl;
    cout << "Average: " << sum / 10.0 << endl;

    return 0;
}
```

2: Write a program that:

- Uses a function to reverse a 1D array.
- Display the array before and after reversal.

```
#include <iostream>
using namespace std;

// Function to reverse array
void reverseArray(int arr[], int size) {
    for(int i = 0; i < size/2; i++) {
        swap(arr[i], arr[size - i - 1]); // swapping elements
```

```
    }
}

int main() {
    int arr[5];

    // Input
    cout << "Enter 5 elements:\n";
    for(int i = 0; i < 5; i++)
        cin >> arr[i];

    // Display original array
    cout << "Original Array: ";
    for(int i = 0; i < 5; i++)
        cout << arr[i] << " ";

    // Reverse function call
    reverseArray(arr, 5);

    // Display reversed array
    cout << "\nReversed Array: ";
    for(int i = 0; i < 5; i++)
        cout << arr[i] << " ";

    return 0;
```

}

3: Write a program to:

• Input two 3×3matrices.

• Perform matrix addition.

• Display the result matrix.

```
#include <iostream>
using namespace std;

int main() {
    int A[3][3], B[3][3], C[3][3];

    // Input matrices
    cout << "Enter elements of Matrix A:\n";
    for(int i = 0; i < 3; i++)
        for(int j = 0; j < 3; j++)
            cin >> A[i][j];

    cout << "Enter elements of Matrix B:\n";
    for(int i = 0; i < 3; i++)
        for(int j = 0; j < 3; j++)
            cin >> B[i][j];

    // Addition
    for(int i = 0; i < 3; i++)
        for(int j = 0; j < 3; j++)
```

```
        C[i][j] = A[i][j] + B[i][j];

    // Display result
    cout << "Resultant Matrix:\n";
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++)
            cout << C[i][j] << " ";
        cout << endl;
    }

    return 0;
}
```

4: Write a program that:

- Stores names of 5 students in a string array.
- Sorts them in alphabetical order.
- Displays the sorted list.

```
#include <iostream>
using namespace std;

int main() {
    string names[5];

    // Input names
    cout << "Enter 5 names:\n";
    for(int i = 0; i < 5; i++)
```

```
        cin >> names[i];

    // Sorting using simple bubble sort
    for(int i = 0; i < 5; i++) {
        for(int j = i + 1; j < 5; j++) {
            if(names[i] > names[j]) {
                swap(names[i], names[j]); // swapping strings
            }
        }
    }

    // Display sorted names
    cout << "Sorted Names:\n";
    for(int i = 0; i < 5; i++)
        cout << names[i] << endl;

    return 0;
}
```

5: Power System Load Analysis (1D Array + Function) In a power system, hourly load demand (in MW) is recorded.

Write a program to:

- Store load data for 24 hours in an array.
- Create functions to calculate: Total load Peak load

```
#include <iostream>
using namespace std;
```

```
// Function to calculate total load
int totalLoad(int arr[], int size) {
    int sum = 0;
    for(int i = 0; i < size; i++)
        sum += arr[i];
    return sum;
}

// Function to find peak load
int peakLoad(int arr[], int size) {
    int max = arr[0];
    for(int i = 1; i < size; i++) {
        if(arr[i] > max)
            max = arr[i];
    }
    return max;
}

int main() {
    int load[24];

    // Input load for 24 hours
    cout << "Enter load for 24 hours:\n";
    for(int i = 0; i < 24; i++)
        cin >> load[i];
```

```
    // Function calls
    cout << "Total Load: " << totalLoad(load, 24) << " MW\n";
    cout << "Peak Load: " << peakLoad(load, 24) << " MW\n";

    return 0;
}
```

6: Design a program to

Store student names (string array) and their marks (1D array).

Use functions to:

Find topper Display pass/fail status

Search a student by name

```
#include <iostream>
using namespace std;

// Function to find topper
int findTopper(int marks[], int size) {
    int maxIndex = 0;
    for(int i = 1; i < size; i++) {
        if(marks[i] > marks[maxIndex])
            maxIndex = i;
    }
    return maxIndex;
}
```

```
// Function to display pass/fail
void passFail(string names[], int marks[], int size) {
  for(int i = 0; i < size; i++) {
    if(marks[i] >= 50)
      cout << names[i] << " : Pass\n";
    else
      cout << names[i] << " : Fail\n";
  }
}

// Function to search student
void searchStudent(string names[], int size, string key) {
  for(int i = 0; i < size; i++) {
    if(names[i] == key) {
      cout << "Student Found at index: " << i << endl;
      return;
    }
  }
  cout << "Student not found\n";
}

int main() {
  string names[5];
  int marks[5];
```

```
    // Input
    cout << "Enter names and marks:\n";
    for(int i = 0; i < 5; i++) {
        cin >> names[i] >> marks[i];
    }

    // Topper
    int topper = findTopper(marks, 5);
    cout << "Topper: " << names[topper] << endl;

    // Pass/Fail
    passFail(names, marks, 5);

    // Search
    string key;
    cout << "Enter name to search: ";
    cin >> key;
    searchStudent(names, 5, key);

    return 0;
}
```

7: Renewable Energy Monitoring System (1D Array + Functions)

A solar power plant records the energy generated (in kWh) every hour for one day.

Write a C++ program to:

• Store energy generation data for 24 hours in a 1D array.

}

Create functions to:

- Calculate total energy generated in a day
- Find the hour with maximum generation
- Calculate average energy generation
- Display all results clearly.

```
#include <iostream>
using namespace std;

// Total energy
float totalEnergy(float arr[], int size) {
    float sum = 0;
    for(int i = 0; i < size; i++)
        sum += arr[i];
    return sum;
}

// Maximum generation hour
int maxHour(float arr[], int size) {
    int index = 0;
    for(int i = 1; i < size; i++) {
        if(arr[i] > arr[index])
            index = i;
    }
    return index;
}
```

```
// Average energy
float averageEnergy(float arr[], int size) {
    return totalEnergy(arr, size) / size;
}

int main() {
    float energy[24];

    // Input
    cout << "Enter energy data for 24 hours:\n";
    for(int i = 0; i < 24; i++)
        cin >> energy[i];

    // Output
    cout << "Total Energy: " << totalEnergy(energy, 24) << " kWh\n";
    cout << "Peak Hour: " << maxHour(energy, 24) << endl;
    cout << "Average Energy: " << averageEnergy(energy, 24) << endl;

    return 0;
}
```

8: Battery Health Monitoring (1D Array + Functions)

In battery management systems, voltage readings are monitored to ensure safe operation.

Write a program to:

- Store 20 voltage readings of a battery in an array.

- Define safe voltage limits (e.g., 3.0V to 4.2V).
- Create functions to:
- Count how many readings are below or above safe limits
- Find the maximum and minimum voltage
- Display a warning message if unsafe readings exist

```
#include <iostream>
using namespace std;

int main() {
   float voltage[20];
   int low = 0, high = 0;

   // Input
   cout << "Enter 20 voltage readings:\n";
   for(int i = 0; i < 20; i++)
      cin >> voltage[i];

   float min = voltage[0], max = voltage[0];

   // Checking limits and finding min/max
   for(int i = 0; i < 20; i++) {
      if(voltage[i] < 3.0)
         low++;
      if(voltage[i] > 4.2)
         high++;
```

```
        if(voltage[i] < min)
            min = voltage[i];
        if(voltage[i] > max)
            max = voltage[i];
    }

    // Output
    cout << "Below safe limit: " << low << endl;
    cout << "Above safe limit: " << high << endl;
    cout << "Minimum Voltage: " << min << endl;
    cout << "Maximum Voltage: " << max << endl;

    if(low > 0 || high > 0)
        cout << "Warning: Unsafe readings detected!\n";

    return 0;
}
```

9: Smart Grid Load Distribution (2D Array + Functions)

In a smart grid, electricity consumption is recorded across multiple regions.

Write a program to:

- Store power consumption (in MW) for 4 regions over 7 days using a 2D array.
- Create functions to:
- Calculate total consumption for each region
- Identify the day with highest total demand

- Compute overall average consumption

```
#include <iostream>
using namespace std;

// Total consumption per region
void regionTotal(int arr[4][7]) {
    for(int i = 0; i < 4; i++) {
        int sum = 0;
        for(int j = 0; j < 7; j++)
            sum += arr[i][j];
        cout << "Region " << i+1 << " Total: " << sum << endl;
    }
}

// Day with highest demand
void highestDay(int arr[4][7]) {
    int maxDay = 0, maxSum = 0;

    for(int j = 0; j < 7; j++) {
        int sum = 0;
        for(int i = 0; i < 4; i++)
            sum += arr[i][j];

        if(sum > maxSum) {
            maxSum = sum;
```

```
            maxDay = j;
        }
    }

    cout << "Highest Demand Day: Day " << maxDay+1 << endl;
}

// Overall average
float overallAverage(int arr[4][7]) {
    int sum = 0;
    for(int i = 0; i < 4; i++)
        for(int j = 0; j < 7; j++)
            sum += arr[i][j];

    return sum / 28.0;
}

int main() {
    int grid[4][7];

    // Input
    cout << "Enter data for 4 regions and 7 days:\n";
    for(int i = 0; i < 4; i++)
        for(int j = 0; j < 7; j++)
            cin >> grid[i][j];
```

```
    // Function calls
    regionTotal(grid);
    highestDay(grid);
    cout << "Overall Average: " << overallAverage(grid) << endl;

    return 0;
}
```

10: Classroom Attendance System (2D Array + Functions)

Teachers often track student attendance daily.

Write a program to:

- Store attendance for 5 students over 7 days using a 2D array (Use 1 for present, 0 for absent).
- Create functions to:
- Calculate total attendance of each student
- Find the student with highest attendance
- Calculate overall class attendance percentage

```
#include <iostream>
using namespace std;

// Total attendance per student
void totalAttendance(int arr[5][7]) {
    for(int i = 0; i < 5; i++) {
        int sum = 0;
        for(int j = 0; j < 7; j++)
            sum += arr[i][j];
```

```
        cout << "Student " << i+1 << ": " << sum << " days present\n";
    }
}

// Student with highest attendance
void highestAttendance(int arr[5][7]) {
    int maxIndex = 0, maxSum = 0;

    for(int i = 0; i < 5; i++) {
        int sum = 0;
        for(int j = 0; j < 7; j++)
            sum += arr[i][j];

        if(sum > maxSum) {
            maxSum = sum;
            maxIndex = i;
        }
    }

    cout << "Top Attendee: Student " << maxIndex+1 << endl;
}

// Overall percentage
float attendancePercentage(int arr[5][7]) {
    int total = 0;
```

```
    for(int i = 0; i < 5; i++)
        for(int j = 0; j < 7; j++)
            total += arr[i][j];

    return (total / 35.0) * 100;
}

int main() {
    int attendance[5][7];

    // Input
    cout << "Enter attendance (1=Present, 0=Absent):\n";
    for(int i = 0; i < 5; i++)
        for(int j = 0; j < 7; j++)
            cin >> attendance[i][j];

    // Function calls
    totalAttendance(attendance);
    highestAttendance(attendance);
    cout << "Class Attendance Percentage: "
         << attendancePercentage(attendance) << "%\n";

    return 0;
}
```